



本科课程论文

(主修专业)

DeepXDE: 一种求解微分方程的 深度学习库

**DeepXDE: A Deep Learning Library for
Solving Differential Equations**

姓名: 卢逸飞

学号: 22920192204031

学院: 信息学院

专业: 人工智能

年级: 2019 级

指导教师: 苏畅 纪荣嵘

2021 年 10 月 26 日

摘要

偏微分方程（PDE）的各种研究是数学研究的热点。其中，求解偏微分方程是一项非常重要和艰巨的任务。由于许多偏微分方程没有解析解，数值方法被广泛用于求解偏微分方程。尽管数值方法已被广泛使用并具有良好的性能，但研究人员仍在寻找求解偏微分方程的新方法。近年来，深度学习在图像分类、自然语言处理等诸多领域取得了巨大的成功。研究表明，深度神经网络具有强大的函数拟合能力，在偏微分方程的研究中具有很大的潜力。在本文中，我介绍了一种用于求解偏微分方程的改进基于物理信息的神经网络（PINN）。PINN 将偏微分方程中包含的物理信息作为正则化项，提高了神经网络的性能。

PINN 算法简单，可以应用于不同类型的偏微分方程，包括积分微分方程、分数阶偏微分方程和随机偏微分方程。此外，从实现的角度来看，PINN 解决逆问题和正问题一样容易。我们提出了一种新的基于残差的自适应细化（RAR）方法来提高 PINN 的训练效率。

我将 PINN 算法与标准有限元方法进行了比较，并且使用了一个用于 PINN 的 Python 库 DeepXDE。具体来说，DeepXDE 可以解决给定初始和边界条件的前向问题，以及给定一些额外测量的逆向问题。DeepXDE 支持基于构造实体几何技术的复杂几何域，并使用户代码紧凑，与数学公式非常相似。更广泛地说，DeepXDE 有助于新兴科学机器学习领域的更快发展。

关键词：深度学习；微分方程；机器学习；DeepXDE；基于物理信息的神经网络

目录

摘要.....	I
1 绪论	1
2 基于物理信息的神经网络的算法与理论	3
2.1 深度神经网络.....	3
2.2 自动微分.....	3
2.3 基于物理信息的神经网络求解偏微分方程.....	5
2.4 PINN 近似理论及误差分析	7
2.5 PINN 与 FEM 算法比较	8
2.6 PINN 求解积分微分方程.....	9
2.7 PINN 求解逆问题.....	10
2.8 基于残差的自适应细化 (RAR)	10
3 DeepXDE 的使用	11
3.1 用法.....	11
3.2 配置.....	12
3.2.1 构造几何体.....	12
3.2.2 定义神经网络.....	13
3.2.3 回调函数.....	13
4 示例	15
4.1 使用基于残差的自适应细化求解 Burgers 方程.....	15
4.2 扩散反应系统的反问题	17
4.3 Volterra IDE.....	17
5 结论	19
参考文献.....	21

1 绪论

偏微分方程（PDE）是研究各种自然现象的重要工具，广泛用于解释各种物理定律^[1]。此外，许多工程和技术问题可以使用偏微分方程进行建模和分析，如尾流湍流、光纤通信、大气污染物扩散等。因此，偏微分方程的进展通常对许多领域都非常重要，例如航空航天、数值天气预报等。目前，偏微分方程和许多其他学科正日益相互联系和加强。因此，研究偏微分方程具有重要意义。然而，偏微分方程研究的一个主要困难是通常不可能获得解析解。因此，相关研究人员提出了各种求解偏微分方程的数值方法，如有限差分法、有限元法、有限体积法等。数值方法极大地促进了偏微分方程的研究。目前，这些方法已得到广泛应用，并在不断改进。与此同时，研究人员还试图开发新的方法和工具来解决偏微分方程。

在过去的 15 年中，神经网络形式的深度学习在各种应用中得到了非常有效的使用，例如计算机视觉和自然语言处理。尽管在这些及相关领域取得了显著的成功，但深度学习尚未在科学计算领域得到广泛应用。近年来，基于神经网络的方法出现在偏微分方程的研究中。例如，Lagaris 等人使用人工神经网络来解决初边值问题^[2]。他们首先构造由两部分组成的试解，第一部分满足初始/边界条件，第二部分为前馈神经网络，然后训练网络满足微分方程。实验结果表明，该方法具有良好的性能。然而，对于高维问题，由于训练集较大，训练时间会增加，这需要通过并行实现等方法来解决^[3,4]。基于 Lagaris 等人的工作，Göküzüm 等人进一步提出了一种基于人工神经网络（ANN）离散化的方法，用于解决均匀化中的周期边值问题^[5]。与 Lagaris 等人相比，Göküzüm 等人使用全局能量势来构建要优化的目标。数值实验表明，该方法可以用较少的神经元数达到合理的物理效果，从而降低了内存需求。然而，该方法仍然面临训练速度慢、过拟合等问题，可以通过退出或正则化来解决。NguyenThanh 等人提出了一种利用能量泛函和深层神经网络研究有限变形超弹性的方法，称为深层能量法（DEM）^[6]。该方法以势能为损失函数，通过最小化能量函数来训练深层神经网络。DEM 在高维问题、不适定问题等方面有着广阔的应用前景，但它也面临着如何添加边界条件、积分技术等问题^[7]。此外，如何更好地构建深层神经网络也是 DEM 深入研究的问题。尽管还存在许多问题，但深度学习方法已逐渐成为求解偏微分方程的一种新方法。

最近,通过深度学习以标准微分形式或积分形式求解偏微分方程已成为科学机器学习名下的一个潜在的新子领域。特别是,我们可以用近似 PDE 解的神经网络代替传统的数值离散化方法^[1]。

为了通过深度学习获得 PDE 的近似解,关键步骤是约束神经网络以最小化 PDE 残差,并且已经提出了几种方法来实现这一点。与传统的基于网格的方法,如有限差分法(FDM)和有限元法(FEM)相比,深度学习可以是一种利用自动微分的无网格方法,并且可以打破维度^[8]。在这些方法中,有些只能应用于特定类型的问题,例如类图像输入域或抛物线偏微分方程。一些研究人员采用 PDE 的变分形式并最小化相应的能量泛函。然而,并不是所有的偏微分方程都可以从一个已知的泛函推导出来,因此也考虑了 Galerkin-type 预测^[9]。或者,可以直接使用强形式的偏微分方程;在这种形式中,可以直接使用自动微分来避免截断误差和变分形式的数值正交误差。这种强形式方法是在中引入的,创造了一种名为基于物理信息的神经网络(PINN)的术语。PINN 的一个吸引人的特性是它可以用于解决逆问题,而前向问题的代码更改最少^[10]。此外, PINN 已进一步扩展到求解积分微分方程(IDE)、分数阶微分方程(FDE)和随机微分方程(SDE)。

本文介绍了在 Python 库 DeepXDE 中实现的各种 PINN 算法,DeepXDE 可用于解决多物理场问题,并支持基于构造立体几何(CSG)技术的复杂几何域,从而避免繁琐且耗时的计算几何^[11]。通过使用 DeepXDE,从实现的角度来看,时间相关的偏微分方程可以像稳定状态一样容易地求解,只需定义初始条件。除了 DeepXDE 的主要工作流程外,用户还可以通过回调函数轻松监控和修改求解过程,例如监控神经网络求解的傅立叶谱,可以揭示神经网络的学习模式^[11]。

第 2 章介绍了 PINN 的算法、近似理论和误差分析,并对 PINN 和 FEM 进行了比较。然后讨论如何使用 PINN 来解决 IDE 和逆问题。此外,可以使用 RAR 方法来提高 PINN 的训练效率。第 3 章介绍了 DeepXDE 的用法及其可定制性。第 4 章通过三个不同的示例展示了 PINN 的功能和 DeepXDE 的友好使用。最后在第 5 章总结这篇论文。

2 基于物理信息的神经网络的算法与理论

本节首先简要概述深度神经网络和自动微分，并介绍用于解决 PDE 的 PINN 算法和理论。然后对 PINN 和 FEM 进行比较，并讨论如何使用 PINN 解决 IDE 和逆问题。接下来提出 RAR，这是一种在训练过程中自适应选择残差点的有效方法。

2.1 深度神经网络

在数学上，深度神经网络是组合函数的特定选择。最简单的神经网络是前馈神经网络 (FNN)，也称为多层感知器 (MLP)，它递归地将线性和非线性变换应用于输入^[1]。尽管在过去的几十年中已经开发了许多不同类型的神经网络，例如卷积神经网络 (CNN) 和循环神经网络 (RNN)，但在本文中考虑使用 FNN，它足以解决大多数 PDE 问题，以及残差神经网络 (ResNet)，这更容易训练深度网络。

令 $N^L(\mathbf{x}): \mathbb{R}^{d_{\text{in}}} \rightarrow \mathbb{R}^{d_{\text{out}}}$ 是一个 L 层神经网络，或是一个 $(L-1)$ 隐藏层神经网络，它在第 l 层有 N_l 个神经元 ($N_0 = d_{\text{in}}, N_L = d_{\text{out}}$)。

分别用 $\mathbf{W}^l \in \mathbb{R}^{N_l \times N_{l-1}}$ 和 $\mathbf{b}^l \in \mathbb{R}^{N_l}$ 表示第 l 层中的权重矩阵和偏置向量，给定一个按对应元素操作的非线性激活函数 σ ，FNN 递归定义如下^[12]：

$$\text{输入层: } N^0(\mathbf{x}) = \mathbf{x} \in \mathbb{R}^{d_{\text{in}}} \quad (1)$$

$$\text{隐藏层: } N^l(\mathbf{x}) = \sigma(\mathbf{W}^l N^{l-1}(\mathbf{x}) + \mathbf{b}^l) \in \mathbb{R}^{N_l} \text{ for } 1 \leq l \leq L-1 \quad (2)$$

$$\text{输出层: } N^L(\mathbf{x}) = \mathbf{W}^L N^{L-1}(\mathbf{x}) + \mathbf{b}^L \in \mathbb{R}^{d_{\text{out}}} \quad (3)$$

常用的激活函数包括 Sigmoid 函数 $1/(1+e^x)$ 、双曲正切函数 $\tanh$ 和整流线性单元 ReLU^[13-15]。

2.2 自动微分

在 PINN 中，需要计算网络输出相对于网络输入的导数。计算导数有四种可能的方法：(1) 手工编码的分析导数；(2) 有限差分法或其他数值近似法；(3) 符号微分；(4) 自动微分 (AD；也称为算法微分)^[16]。在深度学习中，使用反向传播对导数进行评估，反向传播是 AD 的一种专门技术。考虑到神经网络代表组成函数的事实，AD 反复应用链式规则来计算导数。AD 中有两个步

骤：向前传递计算所有变量的值、向后传递计算导数。为了证明 AD，我们考虑只有一个隐含层的模糊神经网络，其中有两个输入 x_1 和 x_2 ，以及一个输出 y [11]：

$$v = -2x_1 + 3x_2 + 0.5 \quad (4)$$

$$h = \tanh v \quad (5)$$

$$y = 2h - 1 \quad (6)$$

用于计算在 $(x_1, x_2) = (2, 1)$ 处的偏导数 $\frac{\partial y}{\partial x_1}$ 和 $\frac{\partial y}{\partial x_2}$ 的 AD 前向传递和后向传递如表 1 所示。

表 1: 用于计算在 $(x_1, x_2) = (2, 1)$ 处的偏导数 $\frac{\partial y}{\partial x_1}$ 和 $\frac{\partial y}{\partial x_2}$ 的 AD 前向传递和后向传递示例

前向传递	后向传递
$x_1 = 2, x_2 = 1$	$\frac{\partial y}{\partial y} = 1$
$v = -2x_1 + 3x_2 + 0.5 = -0.5$	$\frac{\partial y}{\partial h} = \frac{\partial(2h-1)}{\partial h} = 2$
$h = \tanh v \approx -0.462$	$\frac{\partial y}{\partial v} = \frac{\partial y}{\partial h} \frac{\partial h}{\partial v} = \frac{\partial y}{\partial h} \text{sech}^2(v) \approx 1.573$
	$\frac{\partial y}{\partial x_1} = \frac{\partial y}{\partial v} \frac{\partial v}{\partial x_1} = \frac{\partial y}{\partial v} \times (-2) = -3.146$
$y = 2h - 1 = -1.924$	$\frac{\partial y}{\partial x_2} = \frac{\partial y}{\partial v} \frac{\partial v}{\partial x_2} = \frac{\partial y}{\partial v} \times 3 = 4.719$

可以看到，不管输入维数是多少，AD 只需要一个前向过程和一个后向过程来计算所有的偏导数。相反，使用有限差分计算每个偏导数 $\frac{\partial y}{\partial x_i}$ 需要两个函数和一些小的 Δx_i ： $y(x_1, \dots, x_i, \dots, x_{d_{\text{in}}})$ 和 $y(x_1, \dots, x_i + \Delta x_i, \dots, x_{d_{\text{in}}})$ ，因此需要 $d_{\text{in}} + 1$ 个前向通道来评估所有的偏导数。因此，当输入维数较高时，AD 比有限差分法更有效。为了计算 n 阶导数，AD 可以递归应用多次。然而，这种嵌套方法可能会导致效率低下和数值不稳定，因此开发了其他方法，例如泰勒模式

AD。最后，我们注意到，通过 AD，我们可以区分神经网络，因此我们可以处理噪声数据。

2.3 基于物理信息的神经网络求解偏微分方程

本节将介绍基于物理信息的神经网络 (PINN) 和相关设置。传统的神经网络完全基于数据驱动的方法，不考虑数据中包含的物理定律。因此，通常需要大量的数据来训练神经网络以获得合理的模型。相反，PINN 通过强迫网络输出满足相应的偏微分方程，将物理信息引入网络^[1]。具体而言，通过将偏微分方程的正则化加入到损失函数中，使模型在训练过程中考虑物理规律。这种处理使得培训过程需要更少的数据，并加快了培训过程。PINN 不仅可用于求解正问题，即获得偏微分方程的近似解，还可用于求解逆问题，即从训练数据中获得偏微分方程的参数。接下来，介绍了本研究中改进和使用的用于偏微分方程正问题的 PINN。

考虑域上定义的偏微分方程。 Ω 有边界 $\partial\Omega$ ：

$$D(\mathbf{u}(\mathbf{x})) = 0, \mathbf{x} \in \Omega \quad (7)$$

$$B(\mathbf{u}(\mathbf{x})) = 0, \mathbf{x} \in \partial\Omega \quad (8)$$

其中 $\mathbf{u}$ 为未知解， D 为线性或非线性微分算子（例如： $\frac{\partial}{\partial \mathbf{x}}$ 、 $\mathbf{u} \circ \frac{\partial}{\partial \mathbf{x}}$ 、 $\mathbf{u} \circ \frac{\partial^2}{\partial \mathbf{x}^2}$ 等）， B 表示偏微分方程的边界条件（例如：Dirichlet 边界条件、Neumann 边界条件、Robin 边界条件等）。需要注意的一点是，对于包含时间变量的偏微分方程，我们处理 $\mathbf{x}$ 的一个特殊分量，即时域包含在 Ω 。此时，初始条件可以看作是时空域上的一种特殊类型的 Dirichlet 边界条件。

首先，我们构造一个神经网络来逼近偏微分方程的解 $\mathbf{u}(\mathbf{x})$ 。该神经网络用 $\mathbf{u}(\mathbf{x}; \boldsymbol{\theta})$ 表示，它以 $\mathbf{x}$ 为输入，并输出一个与 $\mathbf{u}(\mathbf{x})$ 维数相同的向量。因此，接下来通过定义剩余网络来满足第二部分的要求：

$$f(\mathbf{x}; \boldsymbol{\theta}) := N[\mathbf{u}(\mathbf{x}; \boldsymbol{\theta})] \quad (9)$$

为了建立这种神经网络，需要使用自动微分 (AD)。目前，自动微分技术已广泛集成到许多深度学习框架中，如 Tensorflow 和 PyTorch^[1]。在本文中，对于代理网络 $\mathbf{u}$ ，根据链式规则通过自动微分推导出神经网络。此外，由于网络 f 的参数与网络 $\mathbf{u}$ 相同，因此两个网络都通过最小化损失函数进行训练。图 1 显示了

PINN 的示意图^[11]。

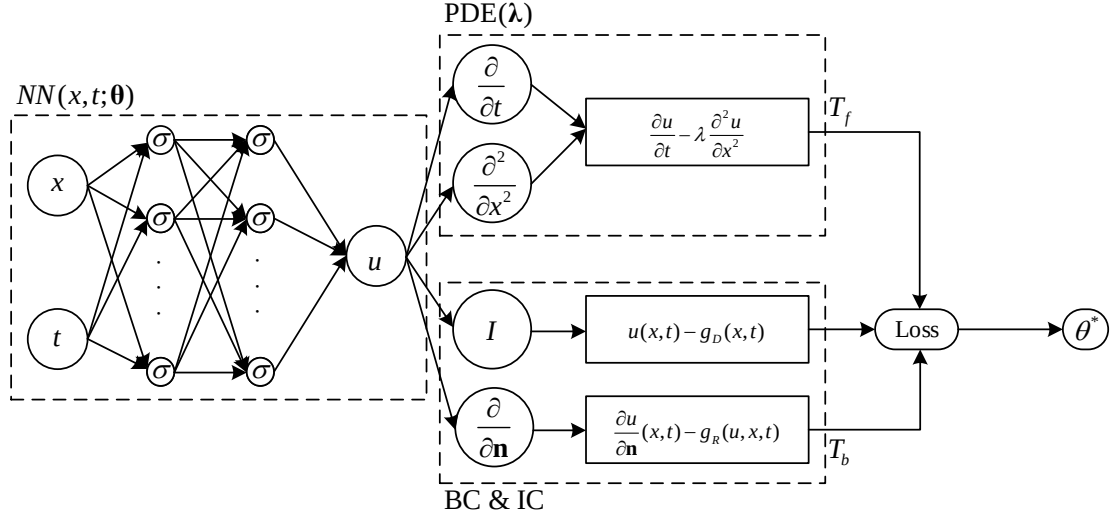


图 1: 基于物理信息的神经网络 (PINN) 示意图

PINN 求解偏微分方程的步骤如下:

- (1) 构造一个带有参数 θ 的神经网络 $u(\mathbf{x}; \theta)$;
- (2) 为方程和边界 (初始条件) 指定两个训练集 T_f 和 T_b 。
- (3) 通过对 PDE 和边界条件残差的加权形式 L^2 求和, 指定损失函数。
- (4) 通过最小化损失函数 $L(\theta; T)$, 训练神经网络以找到最佳参数 θ^* 。

下一步, 我们需要限制神经网络满足偏微分方程和边界条件施加的物理条件。在实践中, 限制一些分散点 (例如, 随机分布点或域中的聚集点), 即大小为 $|T|$ 的训练数据 $T = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{|T|}\}$ 。此外, T 还包括两个集合, $T_f \subset \Omega$ 和 $T_b \subset \partial\Omega$, 分别是域和边界上的点。将集合 T_f 和 T_b 称为 “剩余点”。

为了测量神经网络 u 与约束之间的差异, 我们将损失函数定义为方程残差的平方范数和边界条件的加权求和^[11]:

$$L(\theta; T) = w_f L_f(\theta; T_f) + w_b L_b(\theta; T_b) \quad (10)$$

并且有:

$$L_f(\theta; T_f) = \frac{1}{|T_f|} \sum_{\mathbf{x} \in T_f} \left\| f(\mathbf{x}; \frac{\partial u}{\partial x_1}, \dots, \frac{\partial u}{\partial x_d}; \frac{\partial^2 u}{\partial x_1 \partial x_1}, \dots, \frac{\partial^2 u}{\partial x_d \partial x_d}; \lambda) \right\|_2^2 \quad (11)$$

以及

$$L_b(\boldsymbol{\theta}; T_b) = \frac{1}{|T_b|} \sum_{\mathbf{x} \in T_b} \|B(u, \mathbf{x})\|_2^2 \quad (12)$$

w_f 和 w_b 是权值。损失涉及衍生工具，例如通过 AD 处理的偏导数 $\frac{\partial u}{\partial x_1}$ 或边界

$\frac{\partial u}{\partial \mathbf{n}} = \nabla u \cdot \mathbf{n}$ 处的常规导数。

在最后一步中，通过最小化损失 $L(\boldsymbol{\theta}; T)$ 来搜索一个好的 $\boldsymbol{\theta}$ 的过程被称为“训练”。考虑到损失对于 $\boldsymbol{\theta}$ 是高度非线性和非凸的，通常使用基于梯度的优化器来最小化损失函数，如梯度下降、Adam 和 L-BFGS。注意到光滑的 PDE 解，L-BFGS 可以找到比 Adam 更少迭代次数的好解，因为 L-BFGS 使用损失函数的二阶导数，而 Adam 仅依赖一阶导数^[13,17-19]。然而，对于刚性解决方案，L-BFGS 更可能被困在一个不好的局部最小值。所需的迭代次数在很大程度上取决于问题（例如，解决方案的平滑度^[1,20,21]），为了检查网络是否收敛，我们可以使用 CallbackFunction 监控损失函数或 PDE 残差。另外，可以通过使用自适应激活函数来实现训练的加速，该函数可以去除不好的局部极小值^[4,12,15,22]。

2.4 PINN 近似理论及误差分析

先引入一个定理^[23]：

（神经网络 MLP 模型逼近定理）令 $\mathbf{m}^i \in \mathbb{Z}_+^d, i=1, \dots, s$ ，且 $m = \max_{i=1, \dots, s} |\mathbf{m}^i|$ 。假

设 $\sigma \in C^m(\mathbb{R})$ 并且 σ 不是一个多项式，单隐层神经网络的空间

$$M(\sigma) = \text{span}\{\sigma(\mathbf{w} \cdot \mathbf{x} + b) : \mathbf{w} \in \mathbb{R}^d, b \in \mathbb{R}\}$$

分布于 $C^{\mathbf{m}^1, \dots, \mathbf{m}^s}(\mathbb{R}^d) = \bigcap_{i=1}^s C^{\mathbf{m}^i}(\mathbb{R}^d)$ 。例如：对于任意 $f \in C^{\mathbf{m}^1, \dots, \mathbf{m}^s}(\mathbb{R}^d)$ 、任意 $K \subset \mathbb{R}^d$

和任意 $\varepsilon > 0$ ，存在一个 $g \in M(\sigma)$ ，满足：对于所有的 $\mathbf{k} \in \mathbb{Z}_+^d$ 以及 $\mathbf{k} \leq \mathbf{m}^i$ ，有

$$\max_{\mathbf{x} \in K} |D^{\mathbf{k}} f(\mathbf{x}) - D^{\mathbf{k}} g(\mathbf{x})| < \varepsilon。$$

定理表明，具有足够神经元的前馈神经网络（FNN）可以同时一致地逼近任何函数及其偏导数。让 F 表示可由我们选择的神经网络体系结构表示的所有函数族。解决方案不太可能属于 F 族，我们将 $u_F = \arg \min_{f \in F} \|f - u\|$ 定义为 F 族中接近 u 的最佳函数^[24-26]。

因为只在训练集 T 上训练神经网络，所以将 $u_T = \arg \min_{f \in F} L(f; T)$ 定义为损失为全局最小的神经网络。为简单起见，假设 u 、 u_F 和 u_T 定义良好且唯一。通过最小化损失来查找 u_T 通常在计算上很难，优化器会返回一个近似的解决方案 u_T 。

然后，我们可以将总误差分解为：

$$\varepsilon = \|u_T - u\| \leq \underbrace{\|u - u_T\|}_{\varepsilon_{\text{opt}}} + \underbrace{\|u_T - u_F\|}_{\varepsilon_{\text{gen}}} + \underbrace{\|u_F - u\|}_{\varepsilon_{\text{app}}} \quad (13)$$

近似错误 ε_{app} 测量 u_F 与 u 的近似程度。泛化错误 ε_{gen} 由 T 中剩余点的数量或位置以及 F 系列的容量决定。较大规模的神经网络具有较小的逼近误差，但可能导致较高的泛化误差，这称为偏差-方差均衡。当泛化误差占主导地位时，会发生过度拟合。此外，优化误差 E 从损失函数复杂性和优化设置中选择，如学习率和迭代次数。然而，目前还没有对 PINN 进行误差估计，甚至对监督学习中的三个误差进行量化仍然是一个开放的研究问题^[3,4,9,15]。

2.5 PINN 与 FEM 算法比较

变分问题的解无法解析地找到。因此，可以使用有限元法（FEM）来寻找近似的数值解。方程中的函数和积分可使用有限元空间中的函数进行近似，线性和二次有限元都被考虑在内^[27]。

为了进一步解释 PINN 的概念，逐点比较 PINN 和 FEM：

表 2: PINN 与 FEM 的比较

比较项目	PINN	FEM
基函数	神经网络（非线性）	分段多项式（线性）
参数	权重和偏差	点值
训练点	散点（无网格）	网格点
PDE 嵌入	损失函数	代数系统
参数求解器	基于梯度的优化器	线性求解器
误差	ε_{app} 、 ε_{gen} 和 ε_{opt}	近似、正交误差
误差边界	目前无	部分可用

总结如下^[10,11,27]：

FEM 用一个分段多项式来近似解，点值待定；而 PINN 构造一个神经网络作为代理模型，通过权重和偏差参数化。

FEM 通常需要网格生成，而 PINN 完全无网格，可以使用网格或随机点。

FEM 使用刚度矩阵和质量矩阵将 PDE 转换为代数系统，而 PINN 将 PDE 和边界条件嵌入到损失函数中。

FEM 中的代数系统由线性解算器精确求解，而 PINN 中的网络由基于梯度的优化器学习。

在更基本的层面上，PINN 提供了函数及其导数的非线性近似，而 FEM 代表了线性近似。

2.6 PINN 求解积分微分方程

求解积分微分方程（IDE）仍然使用自动微分技术解析推导整数阶导数，使用经典方法数值逼近（图 2）积分算子^[10]，如高斯求积。因此，我们引入了第四个误差分量：离散化误差 ε_{dis} ，这是由高斯求积近似积分引起的。

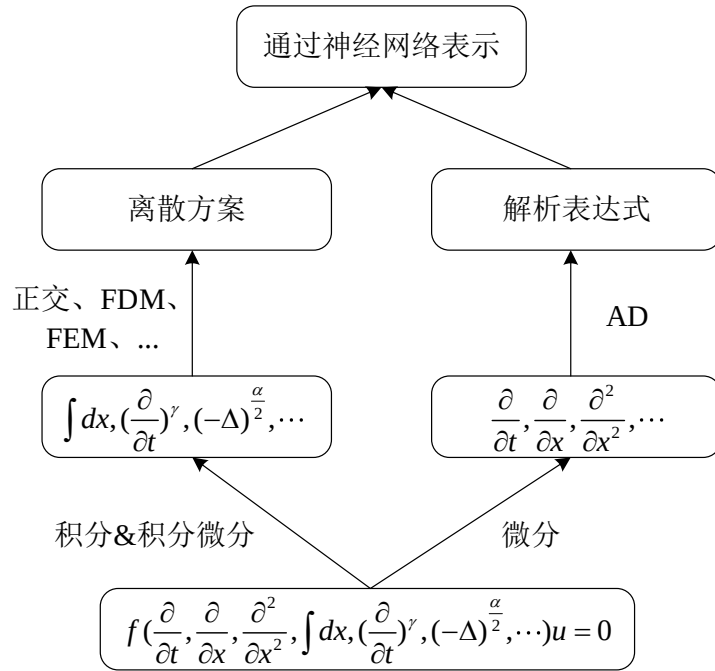


图 2: 用于解决 IDE 的 PINN 算法修改示意图

2.7 PINN 求解逆问题

在逆问题中，有一些未知参数 λ 在(11)，但除了微分方程和边界条件外，还有一些关于点 $T_i \subset \Omega$ 的额外信息: $I(u, \mathbf{x}) = 0, \mathbf{x} \in T_i$ 。

从实现的角度来看，PINN 解决逆问题与解决正问题一样容易^[7,26,28]。解决正问题和逆问题之间的唯一区别在于，在(11)中添加了一个额外的损失项：

$$L(\theta, \lambda; T) = w_f L_f(\theta, \lambda; T_f) + w_b L_b(\theta, \lambda; T_b) + w_i L_i(\theta, \lambda; T_i) \quad (14)$$

此处有：

$$L_i(\theta, \lambda; T_i) = \frac{1}{|T_i|} \sum_{\mathbf{x} \in T_i} \|I(u, \mathbf{x})\|_2^2 \quad (15)$$

然后一起优化 θ 和 λ ，解决方案是 $\theta^*, \lambda^* = \arg \min_{\theta, \lambda} L(\theta, \lambda; T)$ ^[29]。

2.8 基于残差的自适应细化（RAR）

剩余点通常在域中随机分布。这在大多数情况下都很有效，但对于某些具有陡峭梯度的解的偏微分方程来说可能并不有效^[11]。以 Burgers 方程为例；直觉上，应该在尖峰附近放置更多的点，以便很好地捕捉不连续性。然而，对于解未知的问题，设计一个良好的剩余点分布通常很难。给出一种基于残差的自适应细化（RAR）方法，以改善训练过程中残差点分布，概念上类似于 FEM 细化方法

^[11]。RAR 的思想是，将在 PDE 残差 $\left\| f(\mathbf{x}; \frac{\partial u}{\partial x_1}, \dots, \frac{\partial u}{\partial x_d}; \frac{\partial^2 u}{\partial x_1 \partial x_1}, \dots, \frac{\partial^2 u}{\partial x_d \partial x_d}; \lambda) \right\|$ 较大

的位置添加更多残差点，重复添加点，直到平均残差：

$$\varepsilon_r = \frac{1}{V} \int_{\Omega} \left\| f(\mathbf{x}; \frac{\partial u}{\partial x_1}, \dots, \frac{\partial u}{\partial x_d}; \frac{\partial^2 u}{\partial x_1 \partial x_1}, \dots, \frac{\partial^2 u}{\partial x_d \partial x_d}; \lambda) \right\| d\mathbf{x} \quad (16)$$

小于阈值 ε_0 ，其中 V 为 Ω 的体积。

3 DeepXDE 的使用

本节将介绍 DeepXDE 的用法以及如何定制 DeepXDE 以满足新的问题需求。

3.1 用法

与传统的数值方法相比，用 DeepXDE 编写的代码更短、更全面，与数学公式非常相似^[11]。在 DeepXDE 中求解微分方程只需使用内置模块指定问题，包括计算域（几何和时间）、PDE 方程、边界/初始条件、约束、训练数据、神经网络体系结构和训练超参数。工作流程如下^[11]：

（1）使用 `GeometryModule` 指定计算域。

（2）使用 `Tensorflow` 语法指定 PDE。

（3）指定边界和初始条件。

（4）对于时间无关问题和时间相关问题，将几何体、PDE 和边界/初始条件分别组合为 `data.PDE` 或 `data.timepde`。要指定训练数据，我们可以设置特定的点位置，或者只设置点的数量，然后 DeepXDE 将在网格上或随机采样所需数量的点。

（5）使用 `MopsModule` 构建神经网络。

（6）通过结合（4）中的 PDE 问题和（5）中的神经网络来定义模型。

（7）调用 `Model.compile` 设置优化超参数，如优化器和学习率。

（8）调用 `Model.train`，使用参数 `model_restore_path` 从随机初始化或预训练模型训练网络。使用回调监控和修改培训行为。

（9）调用 `Model.predict` 预测不同位置的 PDE 解决方案。

在 DeepXDE 中，内置的基本几何体包括三角形、矩形、多边形、圆盘、长方体和球体。可以使用三个布尔运算从这些基本几何图形构造其他几何图形。这种技术称为构造立体几何（CSG），CSG 支持二维和三维几何图形。

DeepXDE 支持四种标准边界条件，包括 `Dirichlet`（`DirichletBC`）、`Neumann`（`NeumannBC`）、`Robin`（`RobinBC`）和 `periodic`（`periodicBC`），并且可以使用 `OperatorBC` 定义更通用的边界条件。初始条件可以使用 `GIC` 定义。DeepXDE 中有两种类型的神经网络^[11]：前馈神经网络（`maps.FNN`）和残差神经网络（`maps.ResNet`）。还可以方便地选择不同的训练超参数，例如损失类型、度量、

优化器、学习速率计划、初始化和正则化。

除求解微分方程外，DeepXDE 还可用于从多理想数据中逼近函数，并学习非线性算子^[11]。

3.2 配置

DeepXDE 的所有组件都是松散耦合的，因此 DeepXDE 具有良好的结构和高度可配置性。本小节将讨论如何定制 DeepXDE 以满足新的问题需求，例如新的几何结构或网络体系结构。

3.2.1 构造几何体

如上所述，DeepXDE 已经支持 7 种基本几何图形和 CSG 技术。但是，用户仍然可能需要一个新的几何体，而不能在 DeepXDE 中构建。在这种情况下，可以按照程序 1 所示定义新的几何图形。目前，DeepXDE 不准确地支持复杂曲线边界的描述；然而，未来的扩展可能是将非均匀有理基样条曲线（NURBS）用于此类表示。

程序 1: 定义新的几何图形 MyGeometry

```
class MyGeometry(Geometry):
    def inside(self, x):
        """Check if x is inside the geometry."""
    def on_boundary(self, x):
        """Check if x is on the geometry boundary."""
    def boundary_normal(self, x):
        """Compute the unit normal at x for Neumann or Robin boundary
conditions."""
    def periodic_point(self, x, component):
        """Compute the periodic image of x for periodic boundary
condition."""
    def uniform_points(self, n, boundary = True):
        """Compute the equispaced point locations in the geometry."""
    def random_points(self, n, random = "pseudo"):
        """Compute the random point locations in the geometry."""
    def uniform_boundary_points(self, n):
        """Compute the equispaced point locations on the boundary."""
```

```
def random_boundary_points(self, n, random = "pseudo"):
    """Compute the random point locations on the boundary."""
```

3.2.2 定义神经网络

DeepXDE 目前支持两种神经网络：前馈神经网络（`maps.FNN`）和残差神经网络（`maps.ResNet`）。也可以添加新网络，如程序 2 所示。

程序 2：定义新的神经网络 `MyNet`

```
class MyNet(Map):
    @property
    def inputs(self):
        """Return the net inputs."""
    @property
    def outputs(self):
        """Return the net outputs."""
    @property
    def targets(self):
        """Return the targets of the net outputs."""
    def build(self):
        """Construct the network."""
```

3.2.3 回调函数

监控神经网络的训练过程，然后实时进行修改，例如改变学习速率，通常是一种很好的策略。在 DeepXDE 中，这可以通过添加回调函数来实现，下面列举几个已经在 DeepXDE 中实现的回调函数：

- (1) **ModelCheckpoint**: 在特定时期或找到更好的模型后保存模型。
- (2) **OperatorPredictor**: 计算应用于输出的运算符的值。
- (3) **FirstDerivative**: 计算输出相对于输入的一阶导数。
- (4) **MovieDumper**: 在训练过程中转储函数的动画和其傅里叶变换的频谱的动画。

添加其他回调函数非常方便，这些函数将在培训过程的不同阶段调用，见程序 3。

程序 3: 定义新的回调函数 **MyCallback**

```
class MyCallback(Callback):  
    def on_epoch_begin(self):  
        """Called at the beginning of every epoch. """  
    def on_epoch_end(self):  
        """ Called at the end of every epoch."""
```

4 示例

本节使用 PINN 和 DeepXDE 来解决不同的问题。在所有示例中，使用双曲正切函数 $\tanh$ 作为激活函数，表 3 列出了其他超参数。损失函数中的权重 w_f 、 w_b 和 w_i 设置为 1。

表 3: 用于以下 3 个示例的超参数（神经网络缩写为 NN）

样例	NN 深度	NN 宽度	优化器	学习率	迭代次数
1	3	20	Adam, L-BFGS	0.001	10000
2	3	20	Adam	0.001	80000
3	4	20	L-BFGS	—	—

4.1 使用基于残差的自适应细化求解 Burgers 方程

我们首先考虑一维 Burgers 方程^[15]:

$$\begin{cases} \frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}, x \in [-1, 1], t \in [0, 1] \\ u(x, 0) = -\sin(\pi x), u(-1, t) = u(1, t) = 0 \end{cases} \quad (17)$$

设 $\nu = \frac{0.01}{\pi}$ ，首先，随机选择 2500 个点（时空域）作为残差点，然后通过 RAR 在第 2.8 中制定 $m=1$ ， $|S|=100000$ ， $\varepsilon_0 = 0.005$ 。添加新点后的培训程序如表 3 所示。Lu Lu 等人比较了 PINN 解与 RAR 以及基于 2540 个随机选择的训练数据的 PINN 解（图 3）^[11]，并证明了 PINN 与 RAR 能够更好地捕捉不连续性。

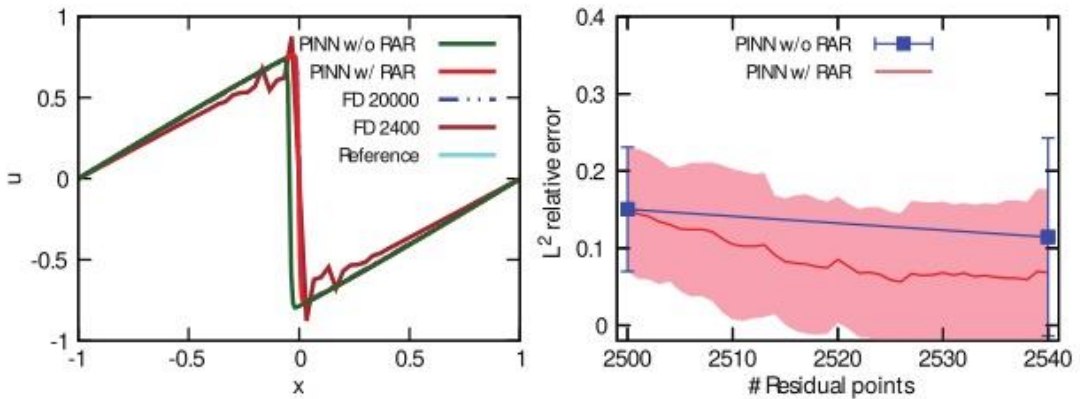


图 3: 左图为各种情况下解的比较，右图为剩余点数量的相对误差

使用 RAR 进一步求解以下二维 Burgers 方程^[15]:

$$\begin{cases} \partial_t u + u \partial_x u + v \partial_y u = \frac{1}{\text{Re}} (\partial_x^2 u + \partial_y^2 u) \\ \partial_t v + u \partial_x v + v \partial_y v = \frac{1}{\text{Re}} (\partial_x^2 v + \partial_y^2 v) \\ x, y \in [0, 1], t \in [0, 1] \end{cases} \quad (18)$$

其中 u 和 v 分别为沿 x 和 y 方向变化速度。此外, Re 是一个无量纲数(雷诺数), 定义为 $\text{Re} = \frac{UL}{\nu}$, 其中 U 和 L 分别表示特征速度和长度, ν 表示流体的运动粘度。

精确解可通过以下方式获得^[15]:

$$\begin{cases} u(x, y, t) = \frac{3}{4} - \frac{1}{4[1 + \exp((-4x + 4y - t) \frac{\text{Re}}{32})]} \\ v(x, y, t) = \frac{3}{4} + \frac{1}{4[1 + \exp((-4x + 4y - t) \frac{\text{Re}}{32})]} \end{cases} \quad (19)$$

在所有边界上使用 Dirichlet 边界条件, Re 设置为 5000。最初, 在时空域中随机采样 200 个残差点, 每个初始和边界条件分别使用 5000 和 1000 个随机点。使用 RAR 只增加了 10 个额外的残差点, 收敛后的残差点总数为 210 个。为了进行比较, 还使用 210 个随机抽样的残差点测试了没有 RAR 的情况。Lu Lu 等人的研究结果显示在图 4 中, 证明了 RAR 的有效性^[11]。

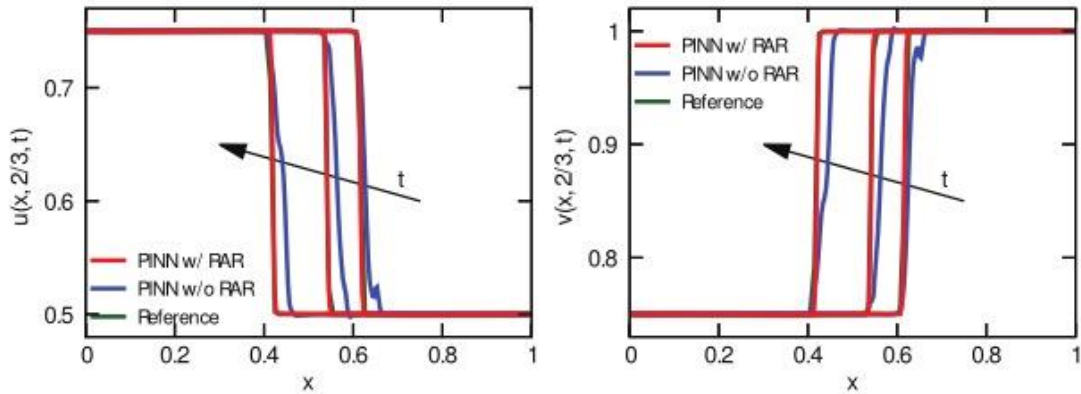


图 4: 二维 Burgers 方程中有 RAR 和无 RAR 的速度剖面

4.2 扩散反应系统的反问题

多孔介质中溶质浓度 C_A 、 C_B 和 C_C ($A+2B \rightarrow C$) 的扩散反应系统可描述为:

$$\left\{ \begin{array}{l} \frac{\partial C_A}{\partial t} = D \frac{\partial^2 C_A}{\partial x^2} - k_f C_A C_B^2 \\ \frac{\partial C_B}{\partial t} = D \frac{\partial^2 C_B}{\partial x^2} - 2k_f C_A C_B^2 \\ x \in [0,1], t \in [0,10] \\ C_A(x,0) = C_B(x,0) = e^{-20x} \\ C_A(0,t) = C_B(0,t) = 1 \\ C_A(1,t) = C_B(1,t) = 0 \end{array} \right. \quad (20)$$

式中, $D=2 \times 10^{-3}$ 是有效扩散系数, k_f 是有效反应速率。由于 D 和 k_f 取决于孔隙结构, 且难以直接测量, 因此我们基于时空域中对 C_A 和 C_B 浓度的 40000 次观测来估算 D 和 k_f 。图 5 中 Lu Lu 等人的实验结果显示的已识别的 $D=1.98 \times 10^{-3}$ 和 $k_f=0.0971$ 与其真实值非常吻合^[11]。

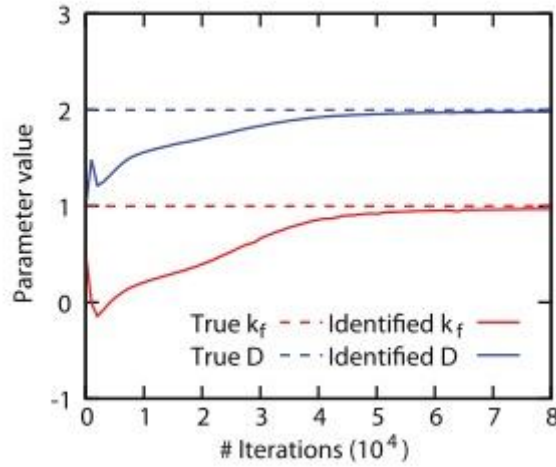


图 5: 扩散反应系统的识别值收敛到真值

4.3 Volterra IDE

考虑域 $[0,5]$ 中 Volterra 型的一阶 IDE:

$$\frac{dy}{dx} + y(x) = \int_0^x e^{t-x} y(t) dt, y(0) = 1 \quad (21)$$

使用精确解 $y(x) = e^{-x} \cosh x$ 。使用 2.6 中的方法求解此 IDE, 并使用 20 次

Gaussian-Legendre 求积近似积分。相对误差为 2×10^{-3} ，解决方案如图 6 所示。

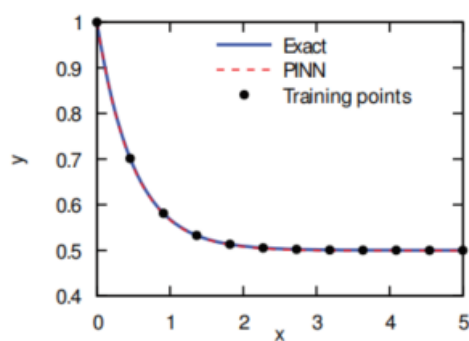


图 6: 求解 Volterra-IDE 的 PINN 算法。蓝色实线是精确解，红色虚线是 PINN 的数值解。使用 12 个等间距的残差点（黑点）。（图片来源 Lu Lu 等人^[11]）

5 结论

本文介绍了求解不同类型偏微分方程 (PDE) 的物理信息神经网络 (PINN) 的算法、逼近理论和误差分析。与传统的数值方法相比, PINN 采用自动微分来处理微分算子, 因此它们是无网格的。与数值微分不同, 自动微分不区分数据, 因此可以容忍噪声数据进行训练。我们还讨论了如何将 PINN 扩展到求解其他类型的微分方程, 如积分微分方程, 以及如何求解反问题。此外提出了一种基于残差的自适应细化 (RAR) 方法, 以改善训练过程中残差点的分布, 从而提高训练效率。正问题和反问题的数值例子验证了 PINN 的有效性和 DeepXDE 的能力。科学机器学习正在成为经典科学计算的一种新的、潜在的强大替代方案。

尽管有上述优点, PINN 仍然有一些局限性。对于正向问题, PINN 目前比有限元速度慢, 但这可以通过离线训练来缓解。对于长时间集成, 还可以使用时间并行方法在多个 GPU 上同时计算较短的时间域。另一个限制是寻找有效的神经网络结构, 这目前是由用户根据经验完成的。然而, 新兴的元学习技术可以用来自动化这种搜索。此外, 虽然在这里强制使用强形式的偏微分方程, 这很容易使用自动微分实现, 但替代的弱、变分形式也可能有效, 尽管它们需要使用正交网格。通过创造性地设计损失函数和引入合适的解空间, 可以跨不同的科学学科对多物理和多尺度问题进行许多其他扩展。例如, 在这里介绍的三个示例中, 仅假设散乱点上的数据。然而, 在地球物理学或生物医学中, 可能有图像和点测量形式的混合数据。在这种情况下, 可以设计一个由一个卷积神经网络和一个共享同一组参数的 PINN 组成的复合神经网络, 并最小化总损耗, 总损耗可以是每个神经网络的多个损耗的加权总和。

参考文献

- [1] Guo Y, Cao X, Liu B, et al. Solving Partial Differential Equations Using Deep Learning and Physical Constraints[J]. Applied Sciences-Basel, 2020, 10(17).
- [2] Lagaris I E, Likas A, Fotiadis D I. Artificial neural networks for solving ordinary and partial differential equations[J]. IEEE transactions on neural networks, 1998, 9(5): 987-1000.
- [3] Teng B, Shi Y, Zhu Q. Solving high-dimensional forward-backward doubly SDEs and their related SPDEs through deep learning[J]. Personal and Ubiquitous Computing, 2021.
- [4] Han J, Jentzen A, Weinan E. Solving high-dimensional partial differential equations using deep learning[J]. Proceedings of the National Academy of Sciences, 2018, 115(34): 8505-8510.
- [5] Göküzüm F S, Nguyen L T K, Keip M-A. An artificial neural network based solution scheme for periodic computational homogenization of electrostatic problems[J]. Mathematical and Computational Applications, 2019, 24(2): 40.
- [6] Nguyen-Thanh V M, Zhuang X, Rabczuk T. A deep energy method for finite deformation hyperelasticity[J]. European Journal of Mechanics-A/Solids, 2020, 80: 103874.
- [7] E W, Han J, Jentzen A. Deep Learning-Based Numerical Methods for High-Dimensional Parabolic Partial Differential Equations and Backward Stochastic Differential Equations[J]. Communications in Mathematics and Statistics, 2017, 5(4): 349-380.
- [8] Colombo A, Perin G. GSR-PDE[J], 2020.
- [9] Li J, Chen Y. A physics-constrained deep residual network for solving the sine-Gordon equation[J]. Communications in Theoretical Physics, 2021, 73(1).
- [10] Pang G, Lu L, Karniadakis G E. fPINN: Fractional physics-informed neural networks[J]. SIAM Journal on Scientific Computing, 2019, 41(4): A2603-A2626.
- [11] Lu L, Meng X, Mao Z, et al. DeepXDE: A deep learning library for solving differential equations[J]. SIAM Review, 2021, 63(1): 208-228.
- [12] Feliu-Faba J, Fan Y, Ying L. Meta-learning pseudo-differential operators with deep neural networks[J]. Journal of Computational Physics, 2020, 408.
- [13] Zhang D, Guo L, Karniadakis G E. Learning in modal space: Solving time-

- dependent stochastic PDEs using physics-informed neural networks[J]. SIAM Journal on Scientific Computing, 2020, 42(2): A639-A665.
- [14] Chen F, Sondak D, Protopapas P, et al. NeuroDiffEq: A Python package for solving differential equations with neural networks[J]. Journal of Open Source Software, 2020, 5(46): 1931.
- [15] Gustafsson T, Mcbain G. scikit-fem: A Python package for finite element assembly[J]. Journal of Open Source Software, 2020, 5(52): 2369.
- [16] Yi H. Efficient architecture for improving differential equations based on normal equation method in deep learning[J]. Alexandria Engineering Journal, 2020, 59(4): 2491-2502.
- [17] Raissi M, Karniadakis G E. Hidden physics models: Machine learning of nonlinear partial differential equations[J]. Journal of Computational Physics, 2018, 357: 125-141.
- [18] Mishra S. A machine learning framework for data driven acceleration of computations of differential equations[J]. arXiv preprint arXiv:1807.09519, 2018.
- [19] Raissi M, Perdikaris P, Karniadakis G E. Machine learning of linear differential equations using Gaussian processes[J]. Journal of Computational Physics, 2017, 348: 683-693.
- [20] Yang Y, Hou M, Luo J, et al. Numerical solution of several kinds of differential equations using block neural network method with improved extreme learning machine algorithm[J]. Journal of Intelligent & Fuzzy Systems, 2020, 38(3): 3445-3461.
- [21] Li J, Chen Y. Solving second-order nonlinear evolution partial differential equations using deep learning*[J]. Communications in Theoretical Physics, 2020, 72(10).
- [22] Aggarwal R, Ugail H, Ieee. On the Solution of Poisson's Equation using Deep Learning[C]. 13th International Conference on Software, Knowledge, Information Management and Applications (SKIMA) / International Workshop on Applied Artificial Intelligence (AI Maldives), 2019.
- [23] Pinkus A. Approximation theory of the MLP model in neural networks[J]. Acta numerica, 1999, 8: 143-195.
- [24] Winovich N, Ramani K, Lin G. ConvPDE-UQ: Convolutional neural networks with quantified uncertainty for heterogeneous elliptic partial differential equations on varied domains[J]. Journal of Computational Physics, 2019, 394: 263-279.

-
- [25] Saha P, Mukhopadhyay S. A Deep Learning Approach for Predicting Spatiotemporal Dynamics From Sparsely Observed Data[J]. Ieee Access, 2021, 9: 64200-64210.
- [26] Hegde P, Heinonen M, Lahdesmaki H, et al. Deep learning with differential Gaussian process flows[C]. 22nd International Conference on Artificial Intelligence and Statistics (AISTATS), 2019.
- [27] Karban P, Pánek D, Orosz T, et al. FEM based robust design optimization with Agros and Ārtap[J]. Computers & Mathematics with Applications, 2021, 81: 618-633.
- [28] Farahmand A-M, Nabi S, Nikovski D N, et al. Deep Reinforcement Learning for Partial Differential Equation Control[C]. American Control Conference (ACC), 2017: 3120-3127.
- [29] Sirignano J, Spiliopoulos K. DGM: A deep learning algorithm for solving partial differential equations[J]. Journal of computational physics, 2018, 375: 1339-1364.